

GWT → Angular 20

AI-Agent Migration Playbook

Target: Angular 20 + Zoneless + Signals-heavy architecture + RxJS + Organization Angular Framework

AI tooling: GitHub Copilot / Copilot Agent

Core principle: Do not translate GWT code line-by-line. Analyze behavior, map responsibilities, design the Angular solution, implement feature-by-feature, and validate against the existing GWT behavior.

1. Target Migration Flow

GWT Application → AI Reverse Engineering → Migration Documentation → Angular Architecture → Angular Foundation → Feature-by-Feature Migration → Automated Testing → GWT vs Angular Validation → Production → GWT Decommission

2. Phase 0 — Establish Migration Rules

Create a migration branch and add:

docs/migration/

docs/architecture/

docs/testing/

.github/copilot-instructions.md

Prompt 00 — Repository Migration Rules

Create and maintain migration rules for a legacy GWT to Angular 20 modernization project. The target is Angular 20, standalone components, zoneless change detection, Signals-heavy state management, RxJS for asynchronous streams, strict TypeScript, Angular Router, Angular HttpClient, and the organization's internal Angular UI framework. Do not perform line-by-line Java/GWT to TypeScript translation. Use existing organization framework components wherever applicable. Do not introduce Angular Material or another UI library when an equivalent organization component exists. Use signal() for state, computed() for derived state, and effect() only for genuine side effects. Use RxJS for HTTP and asynchronous streams. A feature is complete only after implementation, tests, lint, build, zoneless review, organization-framework review, and GWT-vs-Angular behavioral comparison.

3. Phase 1 — Understand the GWT Application

Prompt 01 — Full GWT Repository Analysis

You are a senior software architect specializing in legacy GWT modernization and Angular 20 migrations. We are migrating this existing GWT application to Angular 20. IMPORTANT: Do NOT modify any source code. Do NOT refactor anything. Do NOT generate Angular code yet. Analyze the repository systematically.

Identify:

1. GWT modules
2. entry points
3. startup flow
4. screens/pages
5. panels
6. widgets
7. presenters
8. activities
9. places/history management
10. EventBus and events
11. RPC services

12. asynchronous callbacks
13. DTO/model classes
14. validation
15. business rules
16. navigation
17. authentication/authorization
18. shared components
19. utility classes
20. constants/configuration
21. external libraries
22. backend dependencies
23. APIs/RPC endpoints
24. global/shared state
25. caching
26. error/loading handling

Pay particular attention to hidden coupling.

Determine the responsibility of each class instead of assuming GWT class = Angular component.

Create docs/migration/gwt-application-analysis.md.

4. Phase 2 — Feature Inventory

Prompt 02 — Create GWT Feature Inventory

Using the repository analysis, create a complete feature inventory.

Do NOT modify application source code.

For every user-facing feature identify:

feature name, screen/page, entry point, GWT components, presenter/activity, EventBus events, RPC/API calls, request/response DTOs, business rules, validation, local/shared state, navigation, permissions, loading, errors, empty state, dependencies, complexity, migration risk, and suggested migration order.

Classify each feature as LOW, MEDIUM, HIGH, or VERY HIGH complexity.

Create docs/migration/feature-inventory.md with:

GWT Feature → GWT Classes → API/RPC → Dependencies → Angular Target → Complexity → Risk → Migration Order.

5. Phase 3 — Dependency and Coupling Analysis

Prompt 03 — Dependency Analysis

Analyze the GWT repository specifically for dependencies and coupling.

Do NOT modify source code.

Trace major features as:

UI → Presenter/Activity → EventBus → Service/RPC → DTO → Backend.

Identify shared services, global state, singleton objects, EventBus dependencies, circular dependencies, cross-feature dependencies, shared widgets, validation, utilities, configuration, authentication, authorization, implicit dependencies, static state, and asynchronous dependencies.

Identify the top 20 migration risks and provide mitigation.

Create docs/migration/dependency-map.md and docs/migration/migration-risks.md.

6. Phase 4 — API/RPC Inventory

Prompt 04 — API and GWT RPC Analysis

Analyze all backend communication used by the GWT application.

Do NOT modify source code.

Identify GWT RPC services, methods, requests, responses, exceptions/errors, authentication, authorization, retries, timeouts, caching, pagination, filtering, sorting, uploads/downloads, polling, and long-running operations.

For every operation document:

GWT Client → RPC Interface → Request → Backend Method → Response → Error Handling.

Determine whether Angular can consume the backend directly. If not, identify an adapter/API layer without changing backend code.

Create docs/migration/api-inventory.md.

7. Phase 5 — Define Angular 20 Architecture Using Organization Framework

Prompt 05 — Target Architecture

Act as a senior Angular 20 architect.

Using the migration analysis, feature inventory, dependency map, API inventory, and migration risks, design the target Angular architecture.

Our organization already has an internal Angular UI framework/component library. It is the PRIMARY UI framework.

First inspect and document the available organization components: forms, inputs, selects, buttons, tables/data grids, dialogs, date pickers, notifications, tabs, cards, layout, navigation, loading, errors, validation, icons, typography, design tokens, theming, accessibility utilities, directives, pipes, and shared services.

For each relevant component document:

Organization component → Purpose → Inputs → Outputs → Usage → Applicable GWT UI element.

Requirements: Angular 20, standalone, zoneless, Signals-heavy, RxJS, strict TypeScript, Router, HttpClient, organization framework, organization design system.

UI priority:

1. Organization component
2. Existing application shared component
3. Standard Angular HTML
4. New custom component only when justified.

Do not generate feature code yet.

Create:

docs/architecture/angular-architecture.md

docs/architecture/organization-framework-component-map.md

docs/architecture/state-management.md

docs/architecture/signals-guidelines.md

docs/architecture/zoneless-guidelines.md.

8. Phase 6 — Angular Foundation

Prompt 06 — Build Angular Foundation

Implement the Angular 20 application foundation according to the architecture documents.

Requirements:

Angular 20, standalone architecture, zoneless, strict TypeScript, Router, HttpClient, organization Angular framework, organization design system, accessibility patterns, approved utilities/services, testing and linting.

Before implementation, inspect how the organization framework is imported/configured/used.

Do not use Angular Material or another UI library when an organization equivalent exists.

Do not migrate GWT features yet.

After implementation run tests, lint, and production build. Fix only foundation-related issues.

9. Phase 7 — Select Pilot Feature

Select a MEDIUM-complexity feature with clear UI, clear APIs, and manageable dependencies. Do not start with the largest or most tightly coupled screen. The pilot validates the entire migration process before scaling it.

10. Phase 8 — Analyze One GWT Feature and Map Organization Components

Prompt 07 — Feature Analysis + Organization Component Mapping

We are migrating this GWT feature: [FEATURE NAME]

Do NOT modify source code.

Trace:

User Action → GWT UI → Event Handler → Presenter/Activity → EventBus → Service/RPC → Request → Backend → Response → DTO → UI State.

Identify all relevant files, UI structure, interactions, state, derived state, events, async operations, APIs, models, validation, business rules, navigation, permissions, loading, errors, empty state, and dependencies.

IMPORTANT: For every GWT UI element identify the closest equivalent component from the actual organization Angular framework.

Create:

GWT UI Element → Organization Angular Component → Reason → Inputs → Outputs → Configuration.

If no suitable component exists, explicitly write: NO ORGANIZATION COMPONENT FOUND.

Do not create Angular code.

Create docs/migration/[feature-name]-analysis.md.

11. Phase 9 — Feature Migration Plan

Prompt 08 — Create Feature Migration Plan

Using the feature analysis and architecture documents, create a detailed implementation plan for [FEATURE NAME].

The implementation MUST use organization Angular framework components wherever applicable.

For every UI element define:

GWT UI → Organization Angular component → Angular implementation.

Include models, API service, state, Signals, RxJS, organization components, justified custom components, templates, routing, forms, validation, loading, error, empty state, accessibility, tests, and zoneless verification.

For every proposed custom component answer:

1. Does an equivalent organization component exist?
2. If yes, use it.
3. If no, explain why a custom component is required.

Do not implement.

Create docs/migration/[feature-name]-implementation-plan.md.

12. Phase 10 — Review Migration Plan

Prompt 09 — Review Plan

Review the migration plan for [FEATURE NAME] as a strict senior Angular architect.

Do NOT modify code.

Check GWT-to-Angular mappings, unnecessary components/services/global state, Signal usage, effect() usage, RxJS usage, ZoneJS dependencies, change-detection assumptions, API coupling, business logic duplication, error/loading/validation gaps, tests, accessibility, security, performance, and organization-framework compliance.

Specifically verify that existing organization components are reused and no unnecessary custom/Angular Material/third-party UI components are introduced.

Return problems by severity with file/area, problem, organization alternative, and recommendation.

Then provide the corrected implementation plan.

13. Phase 11 — Agent Implementation

Prompt 10 — Implement Feature Using Organization Framework

Implement the approved migration plan for [FEATURE NAME].

Reference all feature and architecture documents plus .github/copilot-instructions.md.

CRITICAL UI REQUIREMENT:

The screen MUST use the organization's existing Angular framework/component library.

Before creating/modifying UI:

1. Search the organization framework.
2. Determine whether an existing component satisfies the requirement.
3. Reuse it.
4. Follow its official usage pattern.
5. Do not recreate its functionality.

Do NOT use Angular Material, third-party UI libraries, or custom UI components when an equivalent organization component exists.

If no organization component exists, document the fact and justify standard Angular HTML or a custom component.

Use Angular 20, standalone, zoneless, Signals for state, computed() for derived state, RxJS for async, HttpClient for APIs, strict TypeScript.

Preserve business behavior, not GWT implementation details.

Do not modify unrelated features.

After implementation run tests, lint, production build, verify organization component usage, zoneless compatibility, Signal architecture, accessibility, and fix issues.

Report files created/modified, organization components reused, custom components and justification, tests, build, lint, and remaining risks.

14. Phase 12 — Organization Framework Compliance Review

Prompt 11A — Organization Framework Review

Review [FEATURE NAME] as a senior developer expert in our organization's Angular framework.

Do NOT modify code.

Verify UI components, forms, inputs, buttons, dropdowns, tables, dialogs, notifications, loading,

validation, styling, typography, icons, layout, accessibility, utilities, and services.
For every UI element determine:
Current implementation → Organization component that should be used → Correct/Incorrect.
Find duplicated framework functionality, unnecessary custom components, incorrect component APIs, incorrect styling, direct DOM manipulation, hardcoded styles, accessibility violations, third-party UI libraries, and unnecessary Angular Material.
Return findings as CRITICAL/HIGH/MEDIUM/LOW with file, current implementation, organization alternative, reason, and recommended fix.

15. Phase 13 — Signal Architecture Review

Prompt 11 — Signal Review

Review [FEATURE NAME] specifically for Angular Signals architecture.
Do NOT modify code.
Check signal() for state, computed() for derived state, whether effects() are necessary, whether effects can be replaced by computed(), whether RxJS is more appropriate for any flow, mutable state, unnecessary signals, duplicated state, state ownership, rendering implications, and zoneless compatibility.
Return severity, file, problem, and recommendation.

16. Phase 14 — Zoneless Compatibility Review

Prompt 12 — Zoneless Review

Review [FEATURE NAME] for Angular 20 zoneless compatibility.
Do NOT modify code.
Find ZoneJS dependencies, NgZone usage, automatic change-detection assumptions, unnecessary ChangeDetectorRef, non-reactive mutable state, async callbacks that may not update UI, ZoneJS-dependent third-party libraries, lifecycle assumptions, unsafe DOM manipulation, event handling problems, and tests dependent on detectChanges().
Return severity, file, problem, why it matters, and recommended solution.

17. Phase 15 — GWT vs Angular Behavioral Comparison

Prompt 13 — Behavioral Comparison

Compare the original GWT and Angular implementations of [FEATURE NAME].
Do not modify code.
Compare initial screen, user interactions, validation, API requests/parameters, response handling, loading, success, empty/error states, navigation, permissions, business rules, sorting, filtering, pagination, and edge cases.
Create:
Behavior | GWT | Angular | Match? | Difference | Risk
Classify differences as EXPECTED, UNEXPECTED, BUG, or REQUIRES BUSINESS DECISION.
Create docs/testing/[feature-name]-behavior-comparison.md.

18. Phase 16 — Generate Tests

Prompt 14 — Feature Tests

Create comprehensive tests for [FEATURE NAME].
Test behavior rather than implementation details.
Cover initial state, rendering, interactions, successful API response, API errors, loading, empty response, validation, invalid input, navigation, permissions, edge cases, Signal state changes, and RxJS async behavior.
Use the project's configured Angular testing framework.
Do not modify production behavior merely to make tests pass.
Run tests, report failures, fix test problems, and expose genuine production defects.

19. Phase 17 — Senior Code Review

Prompt 15 — Production Code Review

Perform a production-grade code review of [FEATURE NAME].
Do NOT modify code.
Review architecture, Angular 20 practices, standalone components, Signals, computed(), effects(), RxJS, zoneless behavior, TypeScript strictness, performance, memory leaks, security, accessibility, testing,

organization framework compliance, unnecessary custom components, and third-party UI libraries. Return findings as CRITICAL/HIGH/MEDIUM/LOW with file, problem, why, and recommendation.

20. Phase 18 — Fix Approved Findings

Prompt 16 — Fix Review Findings

Fix only the approved findings from the previous reviews for [FEATURE NAME].
Do not introduce unrelated refactoring.
Do not change business behavior.
After changes run unit tests, lint, and production build.
Report files changed, issues fixed, test result, lint result, and build result.

21. Phase 19 — Mark Feature Complete

Prompt 17 — Update Migration Status

Update docs/migration/migration-status.md.
Mark [FEATURE NAME] COMPLETE only if:
GWT behavior analyzed; Angular implementation complete; API integration complete; Signals reviewed; RxJS reviewed; zoneless reviewed; organization framework reviewed; tests complete; lint passes; build passes; behavioral comparison complete; code review complete; and no critical/high issues remain.
If any condition is not satisfied, do not mark COMPLETE.

22. GWT → Angular Mapping Cheat Sheet

GWT Widget → Angular Component

GWT Composite → Angular Component

GWT Presenter → Facade / Service / Component logic depending on responsibility

GWT Activity → Route + Component

GWT Place → Angular Route

GWT EventBus → Signals / RxJS / Outputs depending on behavior

GWT RPC → HttpClient service

GWT AsyncCallback → Observable / Promise

GWT DTO → TypeScript interface/type

GWT HasValue → Signal / FormControl

GWT PopupPanel → Organization Dialog component where available

GWT CellTable → Organization Table/Data Grid where available

GWT History → Angular Router

23. Feature Definition of Done

- UI behavior matches GWT
- Organization framework components used wherever applicable
- No unnecessary custom components
- No unnecessary Angular Material/third-party UI library
- Organization design system followed

- Signals reviewed
- RxJS reviewed
- Zoneless compatibility verified
- API integration complete
- Loading/error/empty states handled
- Validation complete
- Accessibility reviewed
- Tests pass
- Lint passes
- Production build passes
- GWT vs Angular behavior comparison complete
- Code review complete

24. Golden Rules

1. Analyze before coding.
2. Plan before Agent implementation.
3. Migrate one feature at a time.
4. Business behavior is the source of truth—not GWT architecture.
5. Organization framework is the primary UI building block.
6. Signals are for state; RxJS remains appropriate for asynchronous streams.
7. `computed()` is preferred for derived state; avoid unnecessary `effects()`.
8. Every AI-generated change must compile and test.
9. Use AI as coder AND reviewer.
10. Never ask AI to “convert the entire GWT application” in one step.

25. Recommended Execution Loop

Analyze GWT → Map organization components → Create Angular plan → Review plan → Agent implementation → Organization framework review → Signal review → Zoneless review → Tests → GWT vs Angular comparison → Code review → Fix approved issues → Mark complete → Next feature